



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий (ИТ)

Кафедра Математического обеспечения и стандартизации информационных технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 3

по дисциплине

«Технология разработки программных приложений»

Тема: «Системы сборки»

Выполнил:

студент группы ИКБО-20-23

Кузнецов Л. А.

Принял:

Доцент кафедры МОСИТ,
кандидат технических наук,
доцент
Чернов Е. А.

Практическая работа выполнена

«__»_____202__г.

(подпись студента)

«Зачтено»

«__»_____202__ г.

(подпись руководителя)

СОДЕРЖАНИЕ

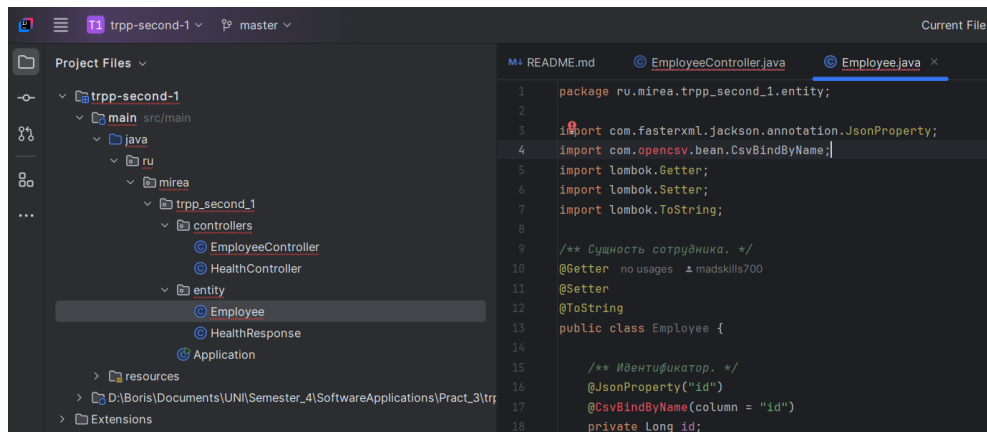
1 ЦЕЛЬ РАБОТЫ	3
2 ХОД РАБОТЫ	4
2.1 Найти отсутствующую зависимость и указать ее в соответствующем блоке в build.gradle, чтобы проект снова начал собираться	4
2.2 В некоторых классах поправить имя пакета	5
2.3 Собрать документацию проекта, найти в ней запросы состояния и сущности по идентификатору.....	6
2.4 Собрать jar со всеми зависимостями (так называемый UberJar), после чего запустить приложение. По умолчанию, сервер стартует на порту 8080.....	8
2.5 Запросить состояние запущенного сервера (GET запрос по адресу http://localhost:8080)	10
2.6 Запросить сущность по идентификатору (GET запрос по адресу: http://localhost:8080/сущность/идентификатор)	11
2.7 В задаче shadowJar добавить к jar-файлу вашу фамилию.....	11
2.8 Выполнить задачу checkstyleMain. Посмотреть сгенерированный отчет. Устранить ошибки оформления кода.....	12
3 КОНТРОЛЬНЫЕ ВОПРОСЫ	14
3.1 Чем компиляция отличается от сборки?	14
3.2 Что такое система сборки?	14
3.3 Что такое репозиторий?.....	14
3.4 Как указать зависимости проекта?	14
3.5 Что такое gradle?	15
3.6 Что такое maven?.....	15
4 ВЫВОД.....	16

1 ЦЕЛЬ РАБОТЫ

Знакомство с системой сборки Gradle. Возможности gradle. Управление зависимостями.

2 ХОД РАБОТЫ

2.1 Найти отсутствующую зависимость и указать ее в соответствующем блоке в build.gradle, чтобы проект снова начал собираться



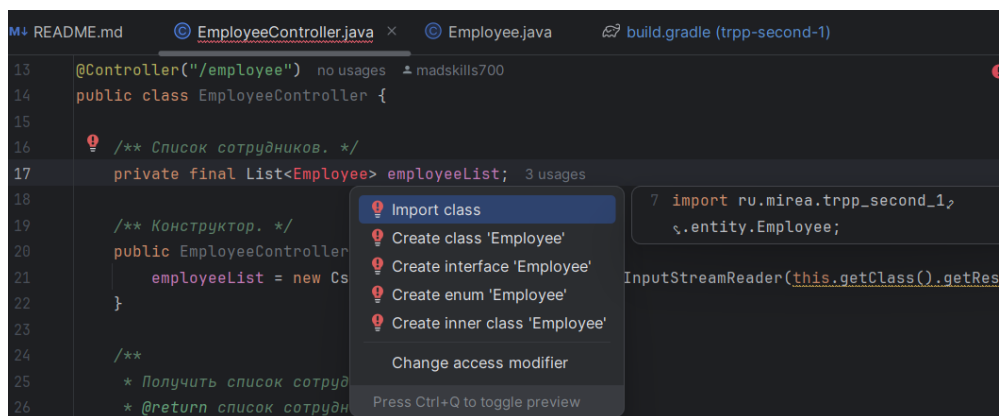
The screenshot shows an IDE with the project structure on the left and the `Employee.java` file open on the right. The project structure includes a `main` directory with `src/main/java` containing `ru`, `mirea`, `trpp_second_1`, `controllers`, `entity`, and `resources`. The `Employee.java` file contains the following code:

```
1 package ru.mirea.trpp_second_1.entity;
2
3 import com.fasterxml.jackson.annotation.JsonProperty;
4 import com.opencsv.bean.CsvBindByName;
5 import lombok.Getter;
6 import lombok.Setter;
7 import lombok.ToString;
8
9 /** Сущность сотрудника. */
10 @Getter no usages madskills700
11 @Setter
12 @ToString
13 public class Employee {
14
15     /** Идентификатор. */
16     @JsonProperty("id")
17     @CsvBindByName(column = "id")
18     private Long id;
```



The screenshot shows the `build.gradle` file for the `trpp-second-1` project. The `dependencies` block is highlighted, showing the following dependencies:

```
23 dependencies {
24     annotationProcessor 'org.projectlombok:lombok:1.18.18'
25     compileOnly 'org.projectlombok:lombok:1.18.18'
26
27     implementation("io.micronaut:micronaut-runtime")
28     implementation("io.micronaut:micronaut-validation")
29     implementation("io.micronaut:micronaut-http-client")
30     implementation("javax.annotation:javax.annotation-api")
31     implementation("org.apache.logging.log4j:log4j-core:2.12.1")
32     implementation("com.opencsv:opencsv:5.9")
33     runtimeOnly("org.apache.logging.log4j:log4j-api:2.12.1")
34     runtimeOnly("org.apache.logging.log4j:log4j-slf4j-impl:2.12.1")
35 }
```



The screenshot shows the `EmployeeController.java` file with a context menu open over the `employeeList` variable. The menu options are:

- Import class
- Create class 'Employee'
- Create interface 'Employee'
- Create enum 'Employee'
- Create inner class 'Employee'
- Change access modifier
- Press Ctrl+Q to toggle preview

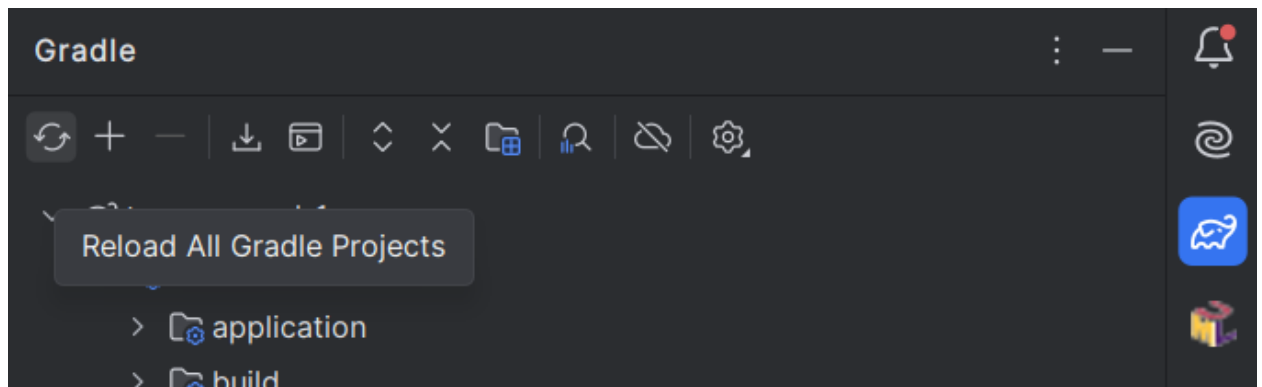
The `EmployeeController.java` file contains the following code:

```
13 @Controller("/employee") no usages madskills700
14 public class EmployeeController {
15
16     /** Список сотрудников. */
17     private final List<Employee> employeeList; 3 usages
18
19     /** Конструктор. */
20     public EmployeeController() {
21         employeeList = new Cs
22     }
23
24     /**
25      * Получить список сотруд
26      * @return список сотруд
```

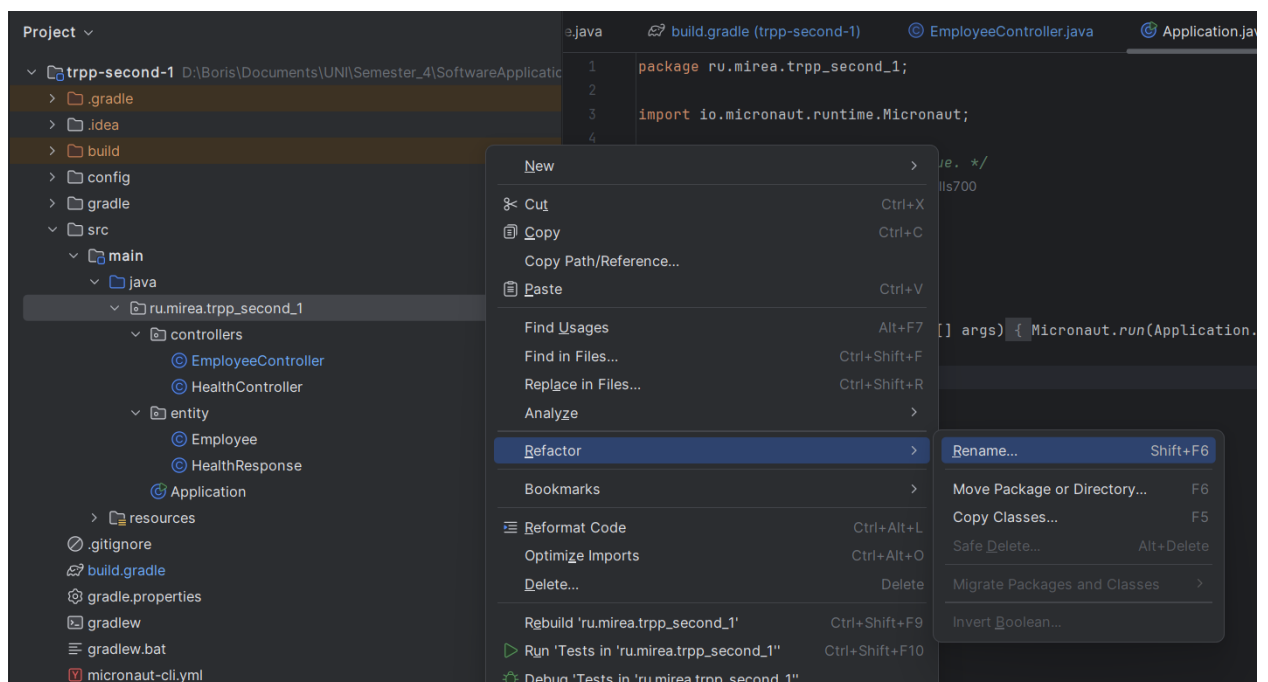
```
/** Список сотрудников. */
private final List<Employee> employeeList; 3 usages

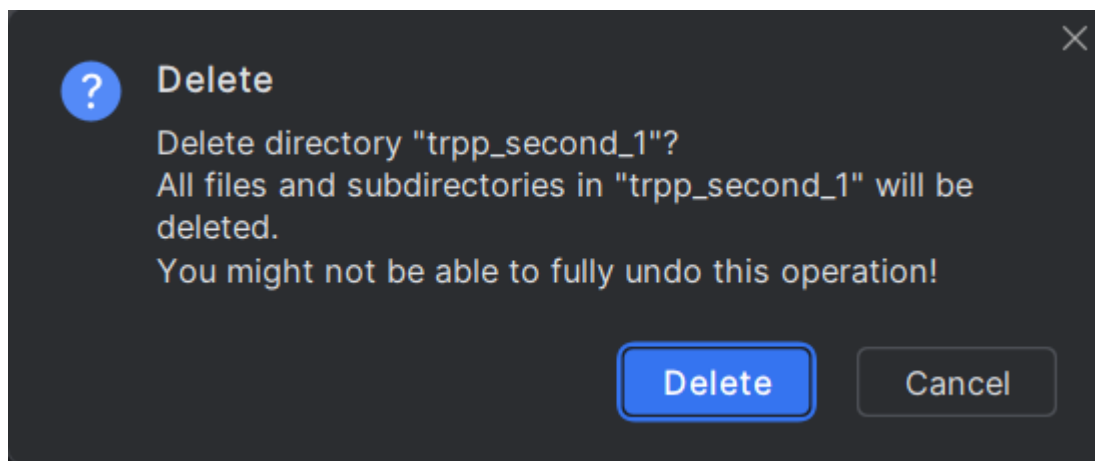
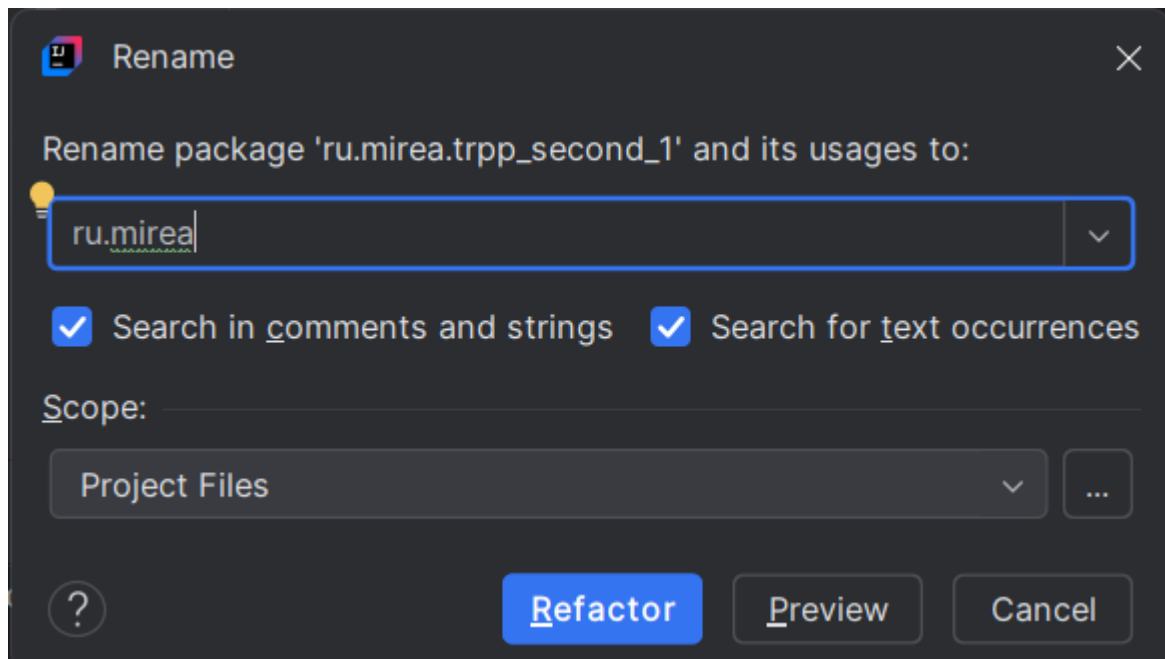
/** Конструктор. */
public EmployeeController() { no usages - maskms700
    employeeList = new CsvToBeanBuilder<Employee>(new InputStreamReader(this.c
})

/**
```



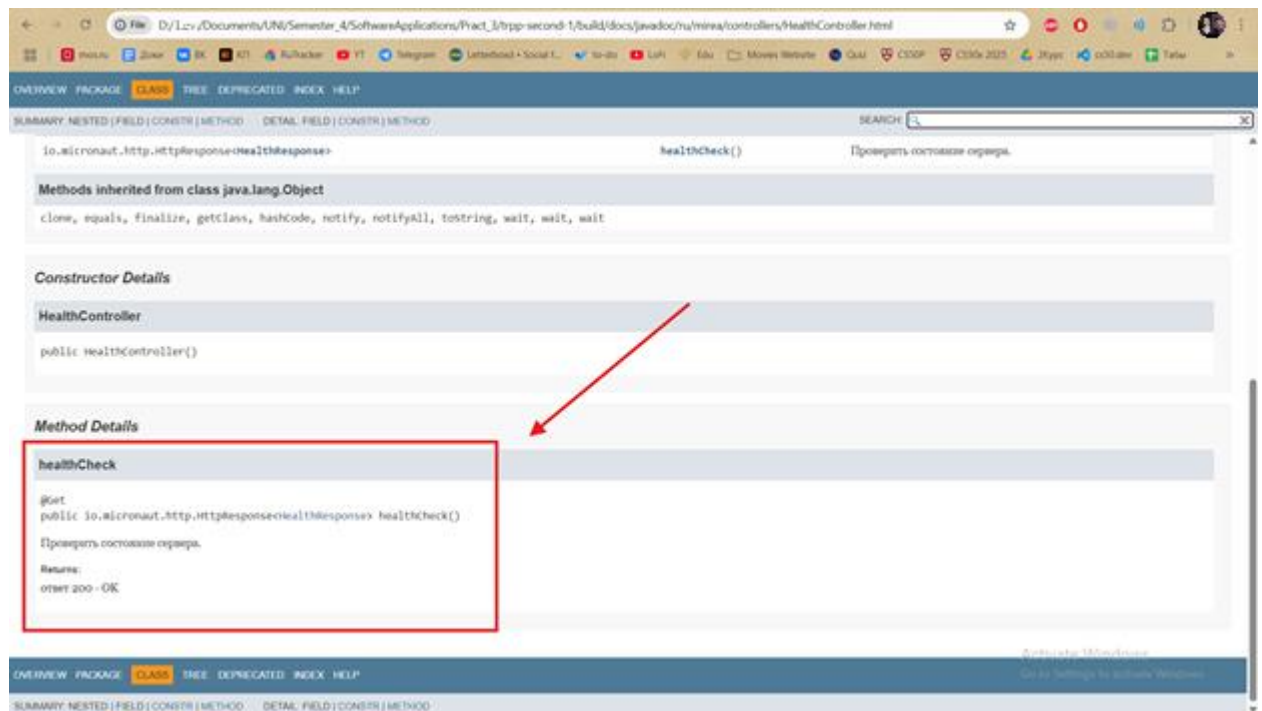
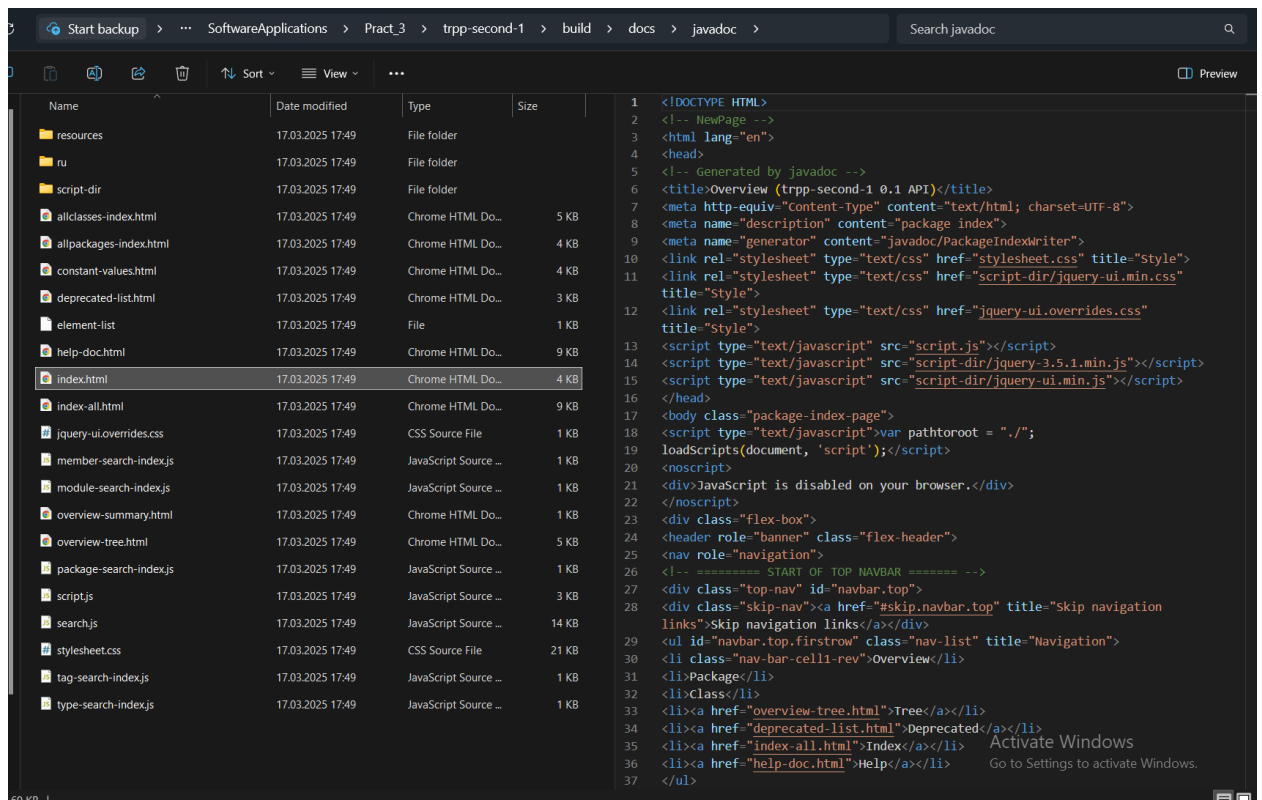
2.2 В некоторых классах поправить имя пакета

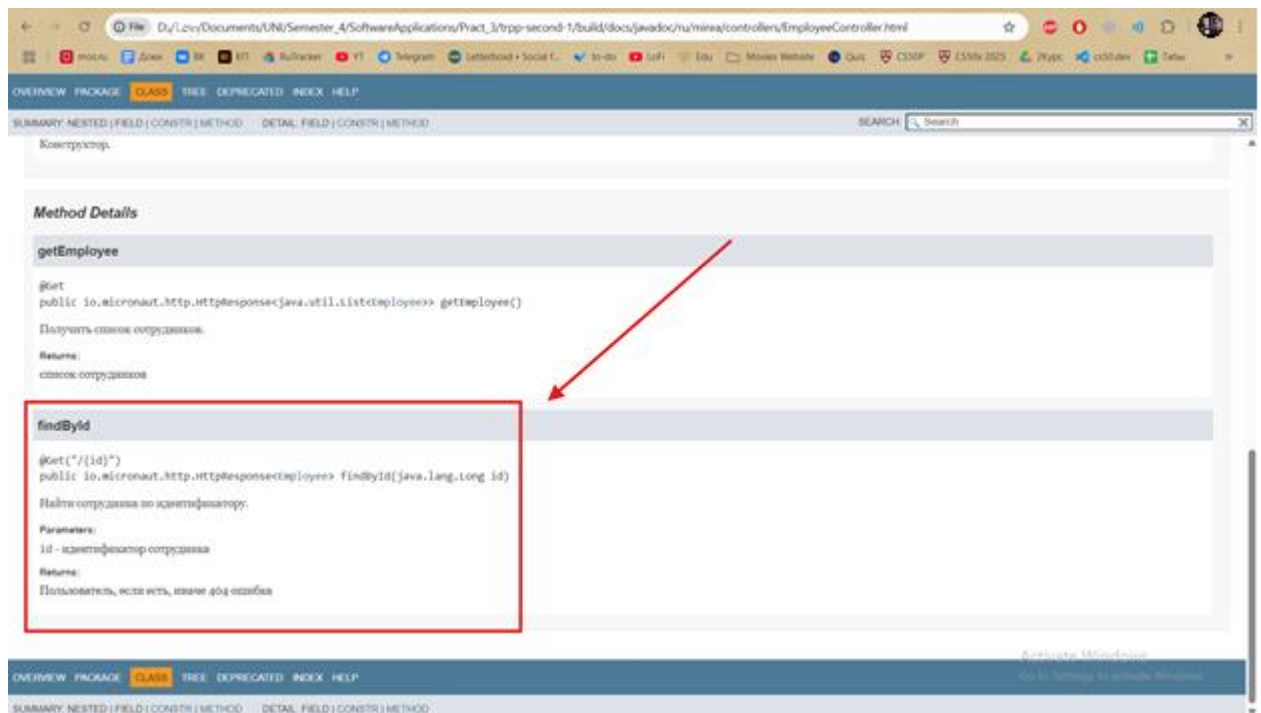




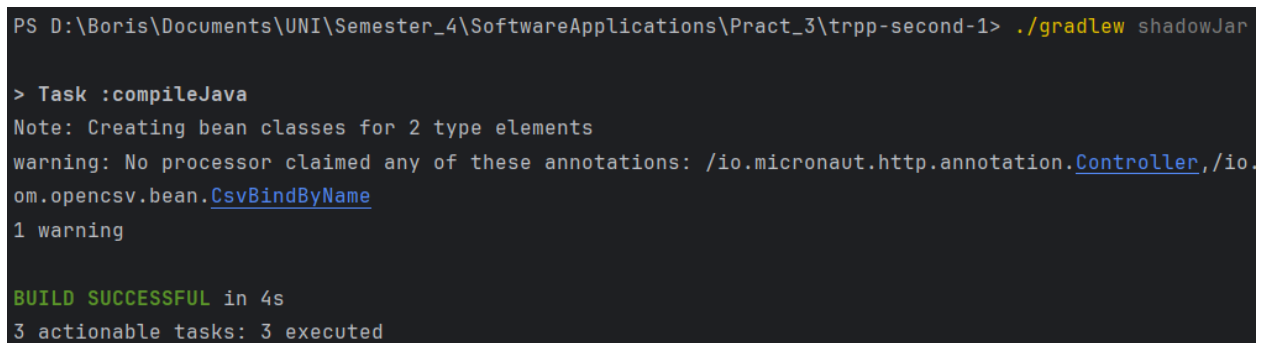
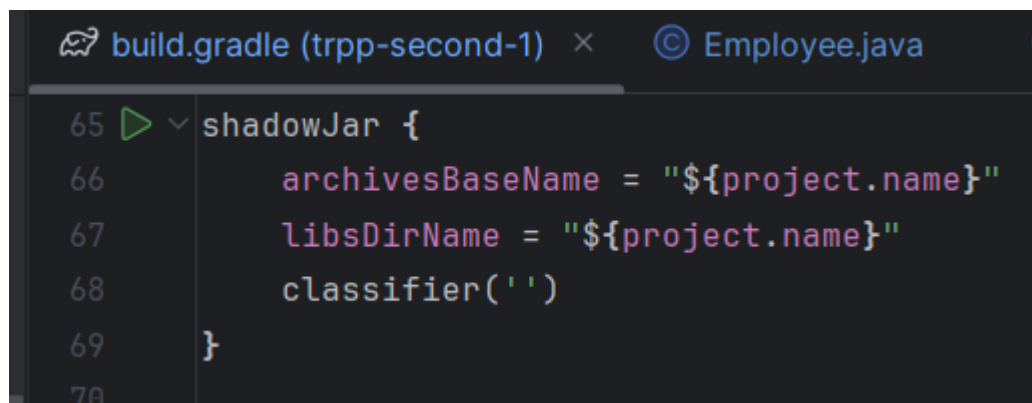
2.3 Собрать документацию проекта, найти в ней запросы состояния и сущности по идентификатору



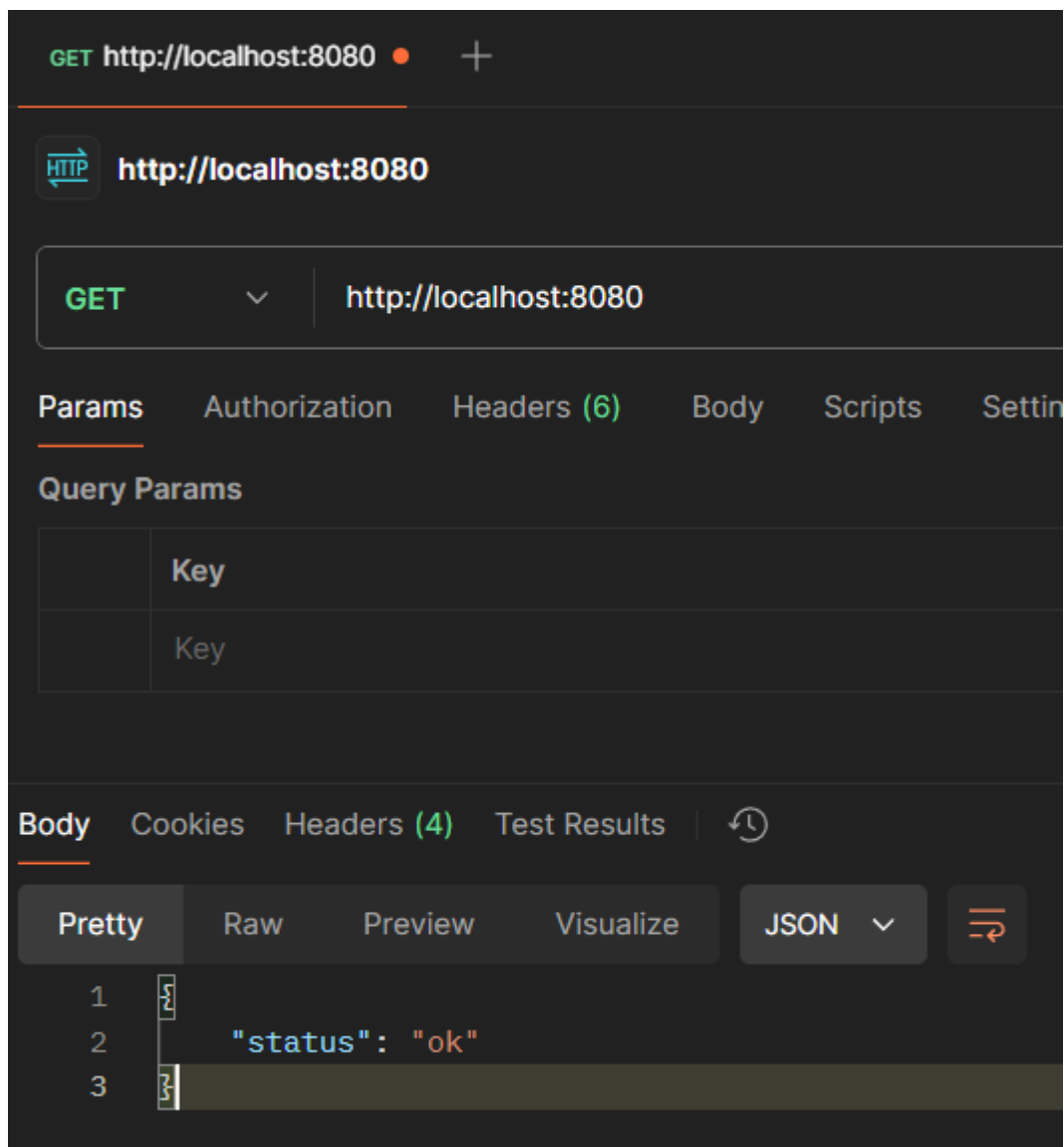




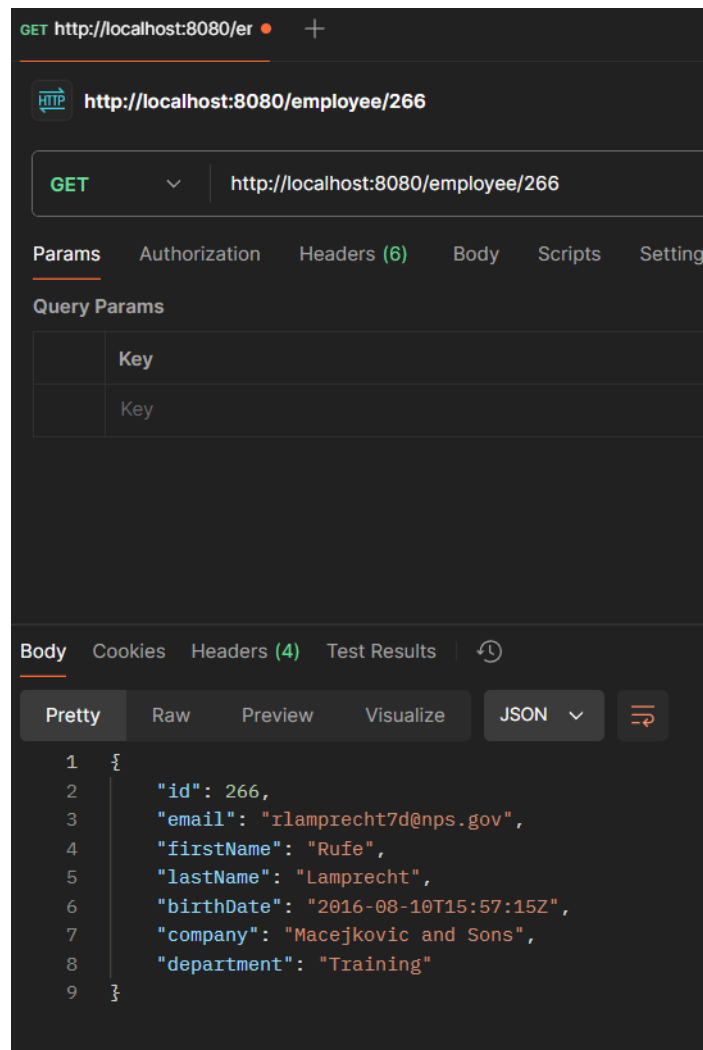
2.4 Собрать jar со всеми зависимостями (так называемый UberJar), после чего запустить приложение. По умолчанию, сервер стартует на порту 8080.



2.5 Запросить состояние запущенного сервера (GET запрос по адресу <http://localhost:8080>)

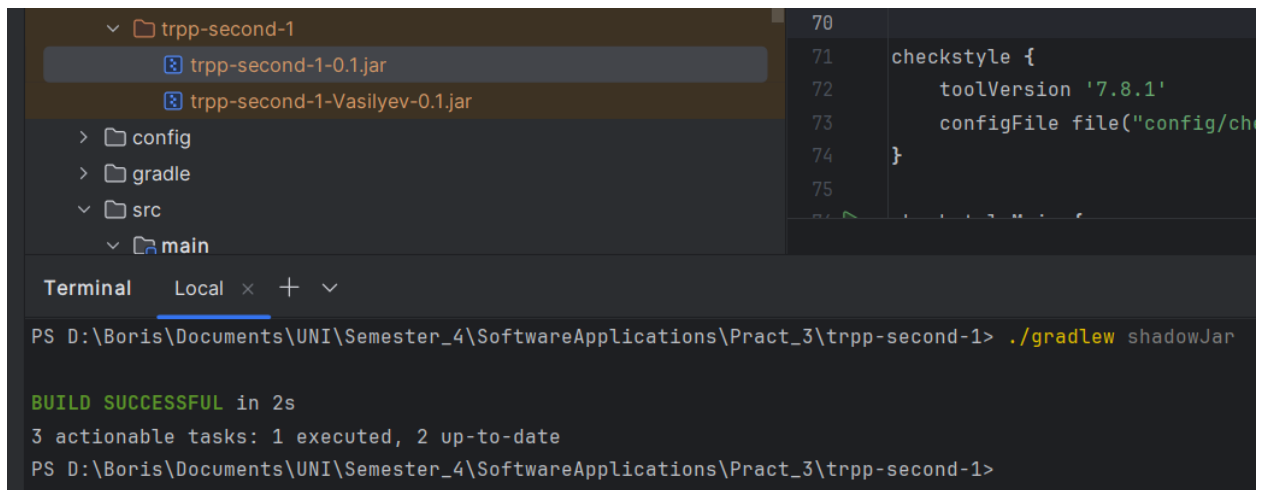


2.6 Запросить сущность по идентификатору (GET запрос по адресу: <http://localhost:8080/сущность/идентификатор>)

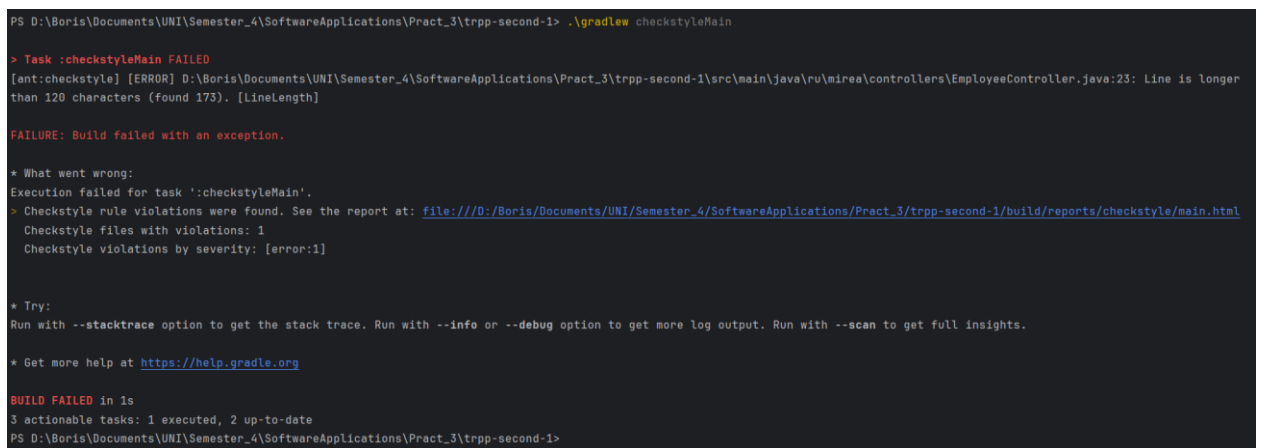


2.7 В задаче shadowJar добавить к jar-файлу вашу фамилию

```
shadowJar {
    archivesBaseName = "${project.name}-Vasilyev"
    libsDirName = "${project.name}"
    classifier('')
}
```



2.8 Выполнить задачу checkstyleMain. Посмотреть сгенерированный отчет. Устранить ошибки оформления кода.



CheckStyle Audit
Designed for use with CheckStyle and Ant

Summary	
Files	Errors
5	1

Files	
Name	Errors
D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\controllers\EmployeeController.java	1
D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\Application.java	0
D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\controllers\HealthController.java	0
D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\entity\Employee.java	0
D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\entity\HealthResponse.java	0

File D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\Application.java	
Error Description	Line
Back to top	

File D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\controllers\EmployeeController.java	
Error Description	Line
Line is longer than 120 characters (found 173)	23
Back to top	

File D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\controllers\HealthController.java	
Error Description	Line
Back to top	

File D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\entity\Employee.java	
Error Description	Line
Back to top	

File D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1\src\main\java\mireal\entity\HealthResponse.java	
Error Description	Line
Back to top	

M+ README.md build.gradle (trpp-second-1) Employee.java EmployeeController.java

```

16 public class EmployeeController {
17     /** список сотрудников. */
18     private final List<Employee> employeeList; 3 usages
19
20
21     /** Конструктор. */
22     public EmployeeController() { no usages madskills700 *
23         employeeList = new CsvToBeanBuilder<Employee>(
24             new InputStreamReader(
25                 this.getClass().getResourceAsStream( name: "/MOCK_DATA.csv")
26             )
27         ).withType(Employee.class).build().parse();
28     }

```

```

PS D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1> .\gradlew checkstyleMain

> Task :compileJava
Note: Creating bean classes for 1 type elements
warning: No processor claimed any of these annotations: /io.micronaut.http.annotation.Controller,/io.micronaut.http.annotation.Get
1 warning

BUILD SUCCESSFUL in 1s
3 actionable tasks: 2 executed, 1 up-to-date
PS D:\Boris\Documents\UNI\Semester_4\SoftwareApplications\Pract_3\trpp-second-1>

```

3 КОНТРОЛЬНЫЕ ВОПРОСЫ

3.1 Чем компиляция отличается от сборки?

Компиляция – это процесс преобразования исходного кода на Java в байт-код, который выполняется виртуальной машиной Java. Сборка включает в себя не только компиляцию, но и обработку зависимостей, генерацию документации, запуск тестов и создание исполняемого файла.

3.2 Что такое система сборки?

Система сборки – это инструмент автоматизации процесса компиляции, тестирования и упаковки программного обеспечения. Она управляет зависимостями, выполняет сценарии сборки и обеспечивает воспроизводимость процесса разработки.

3.3 Что такое репозиторий?

Репозиторий – это хранилище, в котором находятся зависимости, исходный код или артефакты сборки проекта. Он может быть локальным или удалённым и используется для упрощения совместной работы и управления зависимостями.

3.4 Как указать зависимости проекта?

Зависимости проекта указываются в файле конфигурации системы сборки, например, в `build.gradle` для Gradle или `pom.xml` для Maven. В этих файлах прописываются необходимые библиотеки, их версии и источники загрузки

3.5 Что такое gradle?

Gradle — система сборки проектов с открытым исходным кодом, в которой основной упор идет на гибкость и производительность. В настоящее время поддерживаются 2 языка программирования для написания скриптов сборки: groovy и kotlin.

3.6 Что такое maven?

Maven – это более старый инструмент сборки, использующий XML для конфигурации и ориентированный на строгую структуру проектов. Он широко используется благодаря своей простоте и интеграции с экосистемой Java.

4 ВЫВОД

В ходе работы были получены навыки работы с Gradle для управления зависимостями и сборки Java-проекта. Освоены методы поиска и добавления отсутствующих зависимостей в `build.gradle`, а также исправления структуры пакетов в коде. Проведена практика по генерации документации проекта и анализу его API. Выполнена сборка `UberJar` с использованием `shadowJar`, что позволило запустить серверное приложение и проверить его работу через HTTP-запросы. Дополнительно были изучены инструменты статического анализа кода, включая `checkstyleMain`, что позволило выявить и исправить ошибки оформления. Эти знания помогают эффективно управлять сборкой, зависимостями и качеством кода в Java-проектах.